JAVA初級WEBアプリ開発 実践テキスト

Servlet
お問い合わせフォームシステム開発

2025.8.1

【目次】Java 初級 Web アプリ開発 実践テキスト

― 実務で使えるお問い合わせフォームシステム開発 ―

1. プロジェクトの立ち上げ
   1.1 開発環境
   1.2 プロジェクト作成手順（Eclipse 例）
   - Dynamic Web Project 作成
   - Tomcat 10 対応設定（pom.xml 例）
   - パッケージ構造作成
   - JSP と CSS 配置
2. ソースコードと解説
   2.1 Inquiry.java
   - Inquiry.java 全ソース
   - Inquiry.java 全行解説
   2.2 InquiryDAO.java
   - InquiryDAO.java 全ソース
   - InquiryDAO.java 全行解説
   2.3 InquiryServlet.java
   - InquiryServlet.java 全ソース
   - InquiryServlet.java 全行解説
3. JSP ファイルと解説
   3.1 index.jsp
   - index.jsp 全ソース
   - index.jsp 全行解説
   3.2 confirm.jsp
   - confirm.jsp 全ソース
   - confirm.jsp 解説
   3.3 complete.jsp
   - complete.jsp 全ソース
   - complete.jsp 解説
   3.4 inquiry_history.jsp
   - inquiry_history.jsp 全ソース
   - inquiry_history.jsp 解説
4. CSS ファイル（style.css）
   - style.css ソース
5. 動作確認と実行手順
   - サーブレット実行例
   - お問い合わせ履歴の確認

Java 初級 Web アプリ開発 実践テキスト:実務で使えるお問い合わせフォームシステム開発
以下は、本プロジェクトをゼロから立ち上げるところから、すべてのソース、全行解説、JSP、CSS に至るまでを収録してます。

第１章　プロジェクトの立ち上げ

開発環境

- JDK 17 以上
- Apache Tomcat 10 以上
- Eclipse または IntelliJ IDEA（任意）
- Maven または手動でライブラリ管理

プロジェクト作成手順（Eclipse 例）

① Dynamic Web Project を新規作成

- Project Name： inquiry-system
- Target runtime： Tomcat 10.x
- Dynamic web module version： 5.0

---

② jakarta.servlet を使うため、Tomcat 10 対応にする
pom.xml 例（Maven の場合）

```
<dependencies>
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>5.0.0</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

---

③ パッケージ構造を作成

```
src/main/java/com/example/inquiry
```

---

④ WebContent または webapp に JSP と CSS を配置

```
inquiry-system/
├── WebContent/
│   ├── uploads/
│   ├── index.jsp
│   ├── jsp/
│   │   ├── confirm.jsp
│   │   ├── complete.jsp
│   │   └── inquiry_history.jsp
│   └── style.css
└── src/
    └── com/
        └── example/
            └── inquiry/
                ├── Inquiry.java
                ├── InquiryDAO.java
                └── InquiryServlet.java
```

⑤ jar ファイルを WEB-INF/lib に配置

- jakarta.servlet.jsp.jstl-api-2.0.0.jar
- jakarta.servlet.jsp.jstl-2.0.0.jar

Inquiry.java 全ソース

```
package com.example.inquiry;

public class Inquiry {
    private String name;
    private String email;
    private String content;
    private String attachmentFileName;
    private String status;

    public Inquiry() {
        this.status = "新規";
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getContent() {
        return content;
    }

    public void setContent(String content) {
        this.content = content;
    }

    public String getAttachmentFileName() {
        return attachmentFileName;
    }

    public void setAttachmentFileName(String attachmentFileName) {
        this.attachmentFileName = attachmentFileName;
    }
```

```
    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

■Inquiry.java 全行解説

以下は上記ソースの１行ごとの解説だ。

---

package com.example.inquiry;

- クラスの所属パッケージを指定。ファイルの「住所」のようなもの。

---

public class Inquiry {

- Inquiry クラスを定義する行。このクラスは「お問い合わせ」というデータを表現する。

---

private String name;

- ユーザーの名前を格納する変数。

---

private String email;

- ユーザーのメールアドレスを格納する変数。

---

private String content;

- お問い合わせの本文（内容）を格納する変数。

---

private String attachmentFileName;

- ユーザーがアップロードしたファイル名を格納する変数。

---

private String status;

- お問い合わせのステータス（例：「新規」「対応中」「完了」など）を格納する変数。

---

public Inquiry() {

- デフォルトコンストラクタの開始。このクラスのインスタンスを生成するとき呼ばれる。

```
this.status = "新規";
```

- status を「新規」に初期化する。

```
}
```

- コンストラクタの終わり。

getter / setter メソッドの解説。

```
public String getName() {
     return name;
}
```

- name の値を取得する。

```
public void setName(String name) {
    this.name = name;
}
```

- name の値をセットする。

```
public String getEmail() {
    return email;
}
```

- email の値を取得する。

```
public void setEmail(String email) {
    this.email = email;
}
```

- email の値をセットする。

```
public String getContent() {
    return content;
}
```

- content の値を取得する。

```
public void setContent(String content) {
    this.content = content;
}
```

- content の値をセットする。

```
public String getAttachmentFileName() {
    return attachmentFileName;
}
```

- attachmentFileName の値を取得する。

```
public void setAttachmentFileName(String attachmentFileName) {
    this.attachmentFileName = attachmentFileName;
}
```

- attachmentFileName の値をセットする。

```
public String getStatus() {
    return status;
}
```

- status の値を取得する。

```
public void setStatus(String status) {
    this.status = status;
}
```

- status の値をセットする。

```
}
```

- Inquiry クラス定義の終わり。

InquiryDAO.java 全ソース

```java
package com.example.inquiry;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class InquiryDAO {
    private static final List<Inquiry> inquiries = Collections.synchronizedList(new ArrayList<>());

    public void addInquiry(Inquiry inquiry) {
        inquiries.add(inquiry);
    }

    public List<Inquiry> getAllInquiries() {
        return new ArrayList<>(inquiries);
    }

    public Inquiry getInquiryByIndex(int index) {
        if (index >= 0 && index < inquiries.size()) {
            return inquiries.get(index);
        }
        return null;
    }

    public boolean updateInquiryStatus(int index, String newStatus) {
        if (index >= 0 && index < inquiries.size()) {
            inquiries.get(index).setStatus(newStatus);
            return true;
        }
        return false;
    }
}
```

■InquiryDAO.java 全行解説

以下は上記ソースの１行ずつ解説だ。全部やる。

```
package com.example.inquiry;
```

- クラスが属するパッケージを定義する行。Java のファイルは基本的にパッケージに入れるのがルール。

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
```

- 必要な Java 標準ライブラリをインポート。
  - ArrayList：動的配列を作るクラス。
  - Collections：List をスレッドセーフにするために使うユーティリティクラス。
  - List：リストのインターフェース。ArrayList は List の実装。

```
public class InquiryDAO {
```

- DAO クラスの開始。DAO とは「Data Access Object」の略。データ操作を担当するクラス。

```
private static final List<Inquiry> inquiries = Collections.synchronizedList(new ArrayList<>());
```

- メモリ上に問い合わせデータを保持する List を作る。
- synchronizedList でスレッドセーフにしている。
- static final にすることでアプリ全体で共有する「1 つだけのリスト」になる。

```
public void addInquiry(Inquiry inquiry) {
```

- 問い合わせをリストに追加するメソッドの開始。

```
inquiries.add(inquiry);
```

- 引数の Inquiry を List に追加する。

```
}
```

- メソッドの終了。

```
public List<Inquiry> getAllInquiries() {
```

- 全ての Inquiry を取得するメソッドの開始。

```
return new ArrayList<>(inquiries);
```

- inquiries のコピーを返す。
- コピーする理由は、呼び出し元がリストを勝手に書き換えないようにするため。

```
}
```

- メソッドの終了。

```
public Inquiry getInquiryByIndex(int index) {
```

- index 番目の Inquiry を返すメソッドの開始。

```
if (index >= 0 && index < inquiries.size()) {
    return inquiries.get(index);
}
```

- index が範囲内かをチェック。
- OK なら該当する Inquiry を返す。

```
return null;
```

- index が範囲外なら null を返す。

```
}
```

- メソッドの終了。

```
public boolean updateInquiryStatus(int index, String newStatus) {
```

- ステータスを更新するメソッドの開始。
- index と新しいステータス文字列を受け取る。

```
if (index >= 0 && index < inquiries.size()) {
    inquiries.get(index).setStatus(newStatus);
    return true;
}
```

- index が範囲内なら、該当 Inquiry の status を更新する。
- 成功したら true を返す。

```
return false;
```

- index が範囲外なら false を返す。

}

- メソッドの終了。

}

- クラス InquiryDAO の終了。

InquiryServlet.java 全ソース

以下が InquiryServlet.java 全ソース

```
package com.example.inquiry;

import java.io.File;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
import java.util.Random;

import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.MultipartConfig;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import jakarta.servlet.http.Part;

@WebServlet("/inquiry")
@MultipartConfig(fileSizeThreshold = 1024 * 1024 * 2, maxFileSize = 1024 * 1024 * 10, maxRequestSize =
1024 * 1024 * 50)
public class InquiryServlet extends HttpServlet {
    private final InquiryDAO inquiryDAO = new InquiryDAO();
    private static final String UPLOAD_DIRECTORY = "uploads";

    @Override
    public void init() throws ServletException {
        String uploadPath = getServletContext().getRealPath("/uploads");
        File uploadDir = new File(uploadPath);
        if (!uploadDir.exists()) {
            uploadDir.mkdir();
        }
    }
```

```
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
        String action = req.getParameter("action");
        if ("history".equals(action)) {
            req.setAttribute("inquiries", inquiryDAO.getAllInquiries());
            RequestDispatcher rd = req.getRequestDispatcher("/jsp/inquiry_history.jsp");
            rd.forward(req, resp);
        } else {
            generateCaptcha(req);
            RequestDispatcher rd = req.getRequestDispatcher("/index.jsp");
            rd.forward(req, resp);
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException,
IOException {
        req.setCharacterEncoding("UTF-8");
        String action = req.getParameter("action");
        HttpSession session = req.getSession();
        if (action != null && action.equals("complete")) {
            Inquiry inquiry = (Inquiry) session.getAttribute("inquiry");
            if (inquiry != null) {
                inquiryDAO.addInquiry(inquiry);
            }
            session.removeAttribute("inquiry");
            RequestDispatcher rd = req.getRequestDispatcher("/jsp/complete.jsp");
            rd.forward(req, resp);
        } else if (action != null && action.equals("updateStatus")) {
            int index = Integer.parseInt(req.getParameter("index"));
            String newStatus = req.getParameter("newStatus");
            inquiryDAO.updateInquiryStatus(index, newStatus);
            resp.sendRedirect("inquiry?action=history");
        } else {
            String name = req.getParameter("name");
            String email = req.getParameter("email");
            String content = req.getParameter("content");
```

```
            String captchaInput = req.getParameter("captcha");
            String fileName = null;
            try {
                Part filePart = req.getPart("attachment");
                if (filePart != null && filePart.getSize() > 0) {
                    fileName = getFileName(filePart);
                    String uploadPath = getServletContext().getRealPath("") + File.separator +
UPLOAD_DIRECTORY;
                    filePart.write(uploadPath + File.separator + fileName);
                }
            } catch (Exception e) {
                req.setAttribute("errorMessage", "ファイルのアップロード中にエラーが発生しました: " +
e.getMessage());
            }
            Inquiry inquiry = new Inquiry();
            inquiry.setName(name);
            inquiry.setEmail(email);
            inquiry.setContent(content);
            inquiry.setAttachmentFileName(fileName);
            Map<String, String> errors = new HashMap<>();
            if (name == null || name.trim().isEmpty()) {
                errors.put("name", "名前は必須です。");
            }
            if (email == null || email.trim().isEmpty()) {
                errors.put("email", "メールアドレスは必須です。");
            } else if (!email.matches("^[¥¥w.%+-]+@[¥¥w.-]+¥¥.[A-Za-z]{2,6}$")) {
                errors.put("email", "有効なメールアドレスを入力してください。");
            }
            if (content == null || content.trim().isEmpty()) {
                errors.put("content", "内容は必須です。");
            }
            if (!verifyCaptcha(req, captchaInput)) {
                errors.put("captcha", "CAPTCHA が不正です。");
            }
            if (!errors.isEmpty()) {
                req.setAttribute("errors", errors);
                req.setAttribute("inquiry", inquiry);
                generateCaptcha(req);
```

```
            RequestDispatcher rd = req.getRequestDispatcher("/index.jsp");
            rd.forward(req, resp);
        } else {
            session.setAttribute("inquiry", inquiry);
            req.setAttribute("name", name);
            req.setAttribute("email", email);
            req.setAttribute("content", content);
            req.setAttribute("attachmentFileName", fileName);
            RequestDispatcher rd = req.getRequestDispatcher("/jsp/confirm.jsp");
            rd.forward(req, resp);
        }
    }
}

private void generateCaptcha(HttpServletRequest req) {
    Random rand = new Random();
    int num1 = rand.nextInt(10) + 1;
    int num2 = rand.nextInt(10) + 1;
    int answer = num1 + num2;
    req.getSession().setAttribute("captchaAnswer", answer);
    req.setAttribute("captchaQuestion", num1 + " + " + num2 + " = ?");
}

private boolean verifyCaptcha(HttpServletRequest req, String userAnswerStr) {
    HttpSession session = req.getSession(false);
    if (session == null)
        return false;
    Integer captchaAnswer = (Integer) session.getAttribute("captchaAnswer");
    if (captchaAnswer == null || userAnswerStr == null || userAnswerStr.isEmpty()) {
        return false;
    }
    try {
        int userAnswer = Integer.parseInt(userAnswerStr);
        return captchaAnswer.equals(userAnswer);
    } catch (NumberFormatException e) {
        return false;
    }
}
```

```
    private String getFileName(Part part) {
        for (String content : part.getHeader("content-disposition").split(";")) {
            if (content.trim().startsWith("filename")) {
                return content.substring(content.indexOf('=') + 1).trim().replace("¥"", "");
            }
        }
        return null;
    }
}
```

InquiryServlet.java 全行解説

---

package com.example.inquiry;

- このクラスの属するパッケージを指定する。

---

import jakarta.servlet.RequestDispatcher;

- 他の JSP ページへ画面遷移するために使うクラス。

---

import jakarta.servlet.ServletException;

- サーブレットの例外クラス。処理失敗時に投げる。

---

import jakarta.servlet.http.HttpServlet;

- サーブレットを作るための基底クラス。

---

import jakarta.servlet.http.HttpServletRequest;

- HTTP リクエスト情報を扱うクラス。

---

import jakarta.servlet.http.HttpServletResponse;

- HTTP レスポンス情報を扱うクラス。

---

import jakarta.servlet.http.HttpSession;

- セッション管理に使うクラス。

import jakarta.servlet.http.Part;

- ファイルアップロードで送られるデータを表すクラス。

import jakarta.servlet.annotation.MultipartConfig;

- サーブレットでファイルアップロードを受け付けるためのアノテーション。

import java.io.File;

- ファイル操作に使うクラス。

import java.io.IOException;

- 入出力例外を扱うクラス。

import java.util.HashMap;

- Map の実装クラス。エラーメッセージを保持するために使う。

```
import java.util.Map;
```

- Map のインターフェース。

```
import java.util.Random;
```

- CAPTCHA 用のランダムな数字生成に使う。

```
@MultipartConfig(
    fileSizeThreshold = 1024 * 1024 * 2,
    maxFileSize = 1024 * 1024 * 10,
    maxRequestSize = 1024 * 1024 * 50
)
```

- ファイルアップロード時の容量制限を設定する。
  - 2MB を超えたらディスクに書き出し
  - 1 ファイル最大 10MB
  - 全リクエスト最大 50MB

```
public class InquiryServlet extends HttpServlet {
```

- サーブレットクラスの定義。HttpServlet を継承する。

```
private final InquiryDAO inquiryDAO = new InquiryDAO();
```

- DAO のインスタンスを生成。サーブレット全体で共有する。

```
private static final String UPLOAD_DIRECTORY = "uploads";
```

- ファイル保存先ディレクトリ名を定義。

```
@Override
public void init() throws ServletException {
```

- サーブレット初期化メソッド。Tomcat が起動すると最初に呼ばれる。

```
String uploadPath = getServletContext().getRealPath("") + File.separator +
UPLOAD_DIRECTORY;
```

- アップロード用の物理パスを作成する。

```
File uploadDir = new File(uploadPath);
```

- アップロードディレクトリを表す File インスタンスを生成。

```
if (!uploadDir.exists()) {
    uploadDir.mkdir();
}
```

- ディレクトリが存在しなければ作成する。

```
}
```

- init メソッド終了。

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
```

- GET リクエストを処理するメソッド。

```
String action = req.getParameter("action");
```

- リクエストパラメータ "action" を取得。

```
if ("history".equals(action)) {
```

- action が "history" なら履歴画面を表示する処理へ進む。

```
req.setAttribute("inquiries", inquiryDAO.getAllInquiries());
```

- 全問い合わせデータを JSP に渡す。

```
RequestDispatcher rd = req.getRequestDispatcher("/jsp/inquiry_history.jsp");
```

- 履歴画面の JSP のパスを設定。

```
rd.forward(req, resp);
```

- 履歴画面 JSP へフォワード。

```
 } else {
```

- action が "history" 以外の場合の処理。

```
generateCaptcha(req);
```

- CAPTCHA の問題を生成する。

```
RequestDispatcher rd = req.getRequestDispatcher("/index.jsp");
```

- フォーム画面 JSP のパスを設定。

```
rd.forward(req, resp);
```

- フォーム画面へフォワード。

```
  }
 }
```

- doGet メソッド終了。

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
```

- POST リクエストを処理するメソッド。

```
req.setCharacterEncoding("UTF-8");
```

- 文字コードを UTF-8 に設定。

```
String action = req.getParameter("action");
```

- リクエストパラメータ "action" を取得。

```
HttpSession session = req.getSession();
```

- セッションを取得。

---

action = complete の処理

```
if (action != null && action.equals("complete")) {
```

- action が "complete" の場合。

```
Inquiry inquiry = (Inquiry) session.getAttribute("inquiry");
```

- セッションから一時保存した Inquiry を取得。

```
if (inquiry != null) {
      inquiryDAO.addInquiry(inquiry);
}
```

- Inquiry が null でなければ DAO に保存する。

```
session.removeAttribute("inquiry");
```

- セッションから削除する。

```
RequestDispatcher rd = req.getRequestDispatcher("/jsp/complete.jsp");
rd.forward(req, resp);
```

- 完了画面に遷移。

---

action = updateStatus の処理

```
} else if (action != null && action.equals("updateStatus")) {
```

- action が "updateStatus" の場合。

```
int index = Integer.parseInt(req.getParameter("index"));
```

- パラメータ "index" を取得。

```
String newStatus = req.getParameter("newStatus");
```

- パラメータ "newStatus" を取得。

```
inquiryDAO.updateInquiryStatus(index, newStatus);
```

- DAO を通じてステータスを更新。

```
resp.sendRedirect("inquiry?action=history");
```

- 履歴画面へリダイレクト。

その他（初回送信 or バリデーションエラー）

```
} else {
```

- action がない場合の処理。

```
String name = req.getParameter("name");
String email = req.getParameter("email");
String content = req.getParameter("content");
String captchaInput = req.getParameter("captcha");
```

- フォームから送られてきた各パラメータを取得。

ファイルアップロード処理

```
String fileName = null;
```

- ファイル名の初期化。

```
try {
Part filePart = req.getPart("attachment");
```

- ファイルパートを取得。

```
if (filePart != null && filePart.getSize() > 0) {
```

- ファイルが存在し、かつサイズが 0 より大きい場合。

```
fileName = getFileName(filePart);
```

- ファイル名を取り出す。

```
String uploadPath = getServletContext().getRealPath("") + File.separator +
UPLOAD_DIRECTORY;
```

- アップロードパスを生成。

```
filePart.write(uploadPath + File.separator + fileName);
```

- ファイルをサーバ上に書き出す。

```
  }
 } catch (Exception e) {
```

- アップロード中のエラーを捕捉。

```
req.setAttribute("errorMessage", "ファイルのアップロード中にエラーが発生しました:
" + e.getMessage());
 }
```

- エラーメッセージをリクエストに設定。

Inquiry インスタンス作成

```
Inquiry inquiry = new Inquiry();
```

- Inquiry のインスタンス生成。

```
inquiry.setName(name);
inquiry.setEmail(email);
inquiry.setContent(content);
inquiry.setAttachmentFileName(fileName);
```

- 各プロパティに値を設定。

バリデーション

```
Map<String, String> errors = new HashMap<>();
```

- エラーメッセージを格納する Map を用意。

```
if (name == null || name.trim().isEmpty()) {
    errors.put("name", "名前は必須です。");
}
```

- name が空ならエラー。

```
if (email == null || email.trim().isEmpty()) {
    errors.put("email", "メールアドレスは必須です。");
} else if (!email.matches("^[¥¥w.-]+@[¥¥w.-]+¥¥.¥¥w+$")) {
    errors.put("email", "有効なメールアドレスを入力してください。");
}
```

- email が空 or 不正ならエラー。

```
if (content == null || content.trim().isEmpty()) {
    errors.put("content", "内容は必須です。");
}
```

- content が空ならエラー。

```
if (!verifyCaptcha(req, captchaInput)) {
    errors.put("captcha", "CAPTCHA が不正です。");
}
```

- CAPTCHA が正解か判定。

エラーありの場合

```
if (!errors.isEmpty()) {
    req.setAttribute("errors", errors);
    req.setAttribute("inquiry", inquiry);
    generateCaptcha(req);
    RequestDispatcher rd = req.getRequestDispatcher("/index.jsp");
    rd.forward(req, resp);
}
```

- エラーがあれば index.jsp に戻す。

正常時

```
else {
     session.setAttribute("inquiry", inquiry);
```

- セッションに Inquiry を保存。

```
req.setAttribute("name", name);
req.setAttribute("email", email);
req.setAttribute("content", content);
req.setAttribute("attachmentFileName", fileName);
```

- 確認画面に渡すデータをセット。

```
RequestDispatcher rd = req.getRequestDispatcher("/jsp/confirm.jsp");
   rd.forward(req, resp);
     }
  }
}
```

- 確認画面へ遷移。

CAPTCHA 関連

```
private void generateCaptcha(HttpServletRequest req) {
    Random rand = new Random();
    int num1 = rand.nextInt(10) + 1;
    int num2 = rand.nextInt(10) + 1;
    int answer = num1 + num2;
    req.getSession().setAttribute("captchaAnswer", answer);
    req.setAttribute("captchaQuestion", num1 + " + " + num2 + " = ?");
}
```

- CAPTCHA を生成して session に答えを保存する。

```
private boolean verifyCaptcha(HttpServletRequest req, String userAnswerStr) {
    HttpSession session = req.getSession(false);
    if (session == null) return false;
    Integer captchaAnswer = (Integer) session.getAttribute("captchaAnswer");
    if (captchaAnswer == null || userAnswerStr == null ||
userAnswerStr.isEmpty()) {
        return false;
    }
    try {
        int userAnswer = Integer.parseInt(userAnswerStr);
        return captchaAnswer.equals(userAnswer);
    } catch (NumberFormatException e) {
        return false;
    }
}
```

- CAPTCHA の答えをチェックする。

```
private String getFileName(Part part) {
    for (String content : part.getHeader("content-disposition").split(";")) {
        if (content.trim().startsWith("filename")) {
           return content.substring(content.indexOf('=') +
1).trim().replace("¥"", "");
       }
    }
    return null;
 }
}
```

- ファイルアップロードのとき、アップロードされたファイル名を取り出す。

index.jsp 全ソース

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" isELIgnored="false" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:url var="inquiryUrl" value="/inquiry" />
<c:url var="historyUrl" value="/inquiry?action=history" />
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<title>お問い合わせフォーム</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h1>お問い合わせフォーム</h1>
    <form action="${inquiryUrl}" method="post" onsubmit="return validateForm()" enctype="multipart/form-
data">
        <p>
            <label for="name">名前:</label>
            <input type="text" id="name" name="name" value="<c:out value='${inquiry.name}'/>">
            <span class="error-message"><c:out value='${errors.name}' /></span>
        </p>
        <p>
            <label for="email">メールアドレス:</label>
            <input type="email" id="email" name="email" value="<c:out value='${inquiry.email}'/>">
            <span class="error-message"><c:out value='${errors.email}' /></span>
        </p>
        <p>
            <label for="content">内容:</label>
            <textarea id="content" name="content" rows="5" cols="40"><c:out value='${inquiry.content}'
/></textarea>
            <span class="error-message"><c:out value='${errors.content}' /></span>
        </p>
        <p>
            <label for="attachment">添付ファイル:</label>
            <input type="file" id="attachment" name="attachment">
            <span class="error-message"><c:out value='${errorMessage}' /></span>
        </p>
        <p>
            <label for="captcha">スパム対策: <c:out value='${captchaQuestion}' /></label>
            <input type="text" id="captcha" name="captcha" required>
            <span class="error-message"><c:out value='${errors.captcha}' /></span>
        </p>
        <div class="button-group">
            <input type="submit" value="確認">
        </div>
    </form>
```

```
    <!-- ↓ 404 対策：相対パスではなく絶対パスを明示 -->
    <div class="button-group">
        <a href="${historyUrl}" class="button secondary">お問い合わせ履歴を見る</a>
    </div>
</div>

    <script>
function validateForm() {
    let isValid = true;

    const name = document.getElementById('name');
    const email = document.getElementById('email');
    const content = document.getElementById('content');
    const captcha = document.getElementById('captcha');

    document.querySelectorAll('.error-message').forEach(el => el.textContent = '');

    if (name.value.trim() === '') {
        name.nextElementSibling.textContent = '名前は必須です。';
        isValid = false;
    }

    if (email.value.trim() === '') {
        email.nextElementSibling.textContent = 'メールアドレスは必須です。';
        isValid = false;
    } else if (!/^[¥w.%+-]+@[¥w.-]+¥.[A-Za-z]{2,}$/.test(email.value)) {
        email.nextElementSibling.textContent = '有効なメールアドレスを入力してください。';
        isValid = false;
    }

    if (content.value.trim() === '') {
        content.nextElementSibling.textContent = '内容は必須です。';
        isValid = false;
    }

    if (captcha.value.trim() === '') {
        captcha.nextElementSibling.textContent = 'スパム対策の質問に答えてください。';
        isValid = false;
    }

    return isValid;
}
</script>

</body>
</html>
```

■index.jsp 全行解説

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

- JSP ファイルの文字コード設定。

<!DOCTYPE html>

- HTML5 の宣言。

<html lang="ja">

- HTML の開始、言語は日本語。

<head>

- head タグ開始。

<meta charset="UTF-8">

- HTML の文字コードを UTF-8 に指定。

<title>お問い合わせフォーム</title>

- ブラウザのタブに表示されるタイトル。

<link rel="stylesheet" href="style.css">

- 外部 CSS を読み込む。

</head>

- head タグ終了。

<body>

- body 開始。

- デザインを整える div コンテナ。

<h1>お問い合わせフォーム</h1>

- 画面タイトル。

<form action="inquiry" method="post" onsubmit="return validateForm()" enctype="multipart/form-data">

- フォーム開始。送信先は InquiryServlet。

```
<p>
```

- 入力ブロック開始。

```
<label for="name">名前:</label>
```

- 名前のラベル。

```
<input type="text" id="name" name="name" value="<c:out
value='${inquiry.name}'/>">
```

- 名前のテキストボックス。
- JSTL <c:out> で入力値を保持する。

```
<span class="error-message"><c:out value='${errors.name}'/></span>
```

- 名前のエラーメッセージ表示。

以下、email も同様。

```
<label for="email">メールアドレス:</label>
<input type="email" id="email" name="email" value="<c:out
value='${inquiry.email}'/>">
                <span class="error-message"><c:out
value='${errors.email}'/></span>
```

- メールアドレス入力とエラー表示。

```
<label for="content">内容:</label>
<textarea id="content" name="content" rows="5" cols="40"><c:out
value='${inquiry.content}'/></textarea>
                <span class="error-message"><c:out
value='${errors.content}'/></span>
```

- 問い合わせ内容の入力とエラー表示。

```
<label for="attachment">添付ファイル (任意):</label>
<input type="file" id="attachment" name="attachment">
<span class="error-message"><c:out value='${errorMessage}'/></span>
```

- ファイルアップロード。

```
<label for="captcha">スパム対策: ${captchaQuestion}</label>
<input type="text" id="captcha" name="captcha" required>
<span class="error-message"><c:out value='${errors.captcha}'/></span>
```

- CAPTCHA 入力欄。

```
<div class="button-group">
    <input type="submit" value="確認">
</div>
```

- 送信ボタン。

```
<div class="button-group">
    <a href="inquiry?action=history" class="button secondary">お問い合わせ履歴を見る</a>
</div>
```

- 履歴画面へのリンク。

JavaScript 部分

```
<script>
```

- スクリプト開始。

```
function validateForm() {
    let isValid = true;
```

- JS のバリデーション関数開始。

```
const name = document.getElementById('name');
const email = document.getElementById('email');
const content = document.getElementById('content');
const captcha = document.getElementById('captcha');
```

- フォーム要素取得。

```
document.querySelectorAll('.error-message').forEach(el => el.textContent = '');
```

- エラーメッセージ初期化。

各入力チェック：

```
if (name.value.trim() === '') {
    document.querySelector('#name + .error-message').textContent = '名前は必須です。';
    isValid = false;
}
```

```
if (email.value.trim() === '') {
    document.querySelector('#email + .error-message').textContent = 'メールアドレスは必須です。';
    isValid = false;
} else if (!/^[¥w.-]+@[¥w.-]+¥.¥w+$/.test(email.value)) {
    document.querySelector('#email + .error-message').textContent = '有効なメールアドレスを入力してください。';
    isValid = false;
}
```

```
if (content.value.trim() === '') {
   document.querySelector('#content + .error-message').textContent = '内容は必須です。';
     isValid = false;
}
```

```
if (captcha.value.trim() === '') {
    document.querySelector('#captcha + .error-message').textContent = 'スパム対策の質問に答えてください。';
    isValid = false;
}
```

```
return isValid;
}
```

- バリデーション結果を返す。

```
</script>
```

- スクリプト終了。

</body>
</html>

- HTML 終了。

confirm.jsp 全ソース

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <title>お問い合わせ内容確認</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="container">
        <h1>お問い合わせ内容確認</h1>
        <p>以下の内容でよろしいですか？</p>

        <div class="display-info">
            <p><strong>名前:</strong> <%= request.getAttribute("name") %></p>
            <p><strong>メールアドレス:</strong> <%= request.getAttribute("email") %></p>
            <p><strong>内容:</strong></p>
            <pre><%= request.getAttribute("content") %></pre>
            <p><strong>添付ファイル:</strong>
                <% String attachmentFileName = (String) request.getAttribute("attachmentFileName"); %>
                <% if (attachmentFileName != null && !attachmentFileName.isEmpty()) { %>
                    <%= attachmentFileName %>
                <% } else { %>
                    なし
                <% } %>
            </p>
        </div>

        <form action="../inquiry" method="post">
            <input type="hidden" name="action" value="complete">
            <div class="button-group">
                <input type="submit" value="送信">
                <a href="../inquiry" class="button secondary">戻る</a>
            </div>
        </form>
    </div>
</body>
</html>
```

■confirm.jsp 解説

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

- JSP のファイル設定。
- 出力コンテンツタイプと文字エンコーディングを UTF-8 に指定。

<!DOCTYPE html>

- HTML5 の宣言。

<html lang="ja">

- HTML の開始。言語は日本語。

<head>

- head タグの開始。

<meta charset="UTF-8">

- HTML の文字コード指定。

<title>お問い合わせ内容確認</title>

- ブラウザのタブに表示されるページタイトル。

<link rel="stylesheet" href="../style.css">

- CSS ファイルを読み込む。../ は jsp/ フォルダから 1 つ上に戻る。

</head>

- head タグ終了。

<body>

- body 開始。

- CSS の中央寄せレイアウト用コンテナ。

<h1>お問い合わせ内容確認</h1>

- ページ見出し。

以下の内容でよろしいですか？

- 確認メッセージ。

```
<div class="display-info">
```

- 表示エリア開始。

名前の表示

```
<p><strong>名前:</strong> <%= request.getAttribute("name") %></p>
```

- name を動的に表示する。
- <%= ... %> は JSP の式タグ（スクリプトレット）。リクエストスコープから値を取り出す。

メールアドレスの表示

```
<p><strong>メールアドレス:</strong> <%= request.getAttribute("email") %></p>
```

- email を表示する。

内容の表示

```
<p><strong>内容:</strong></p>
<pre><%= request.getAttribute("content") %></pre>
```

- content を <pre> で表示。改行をそのまま反映。

添付ファイルの表示

```
<p><strong>添付ファイル:</strong>
```

- 添付ファイル欄の開始。

```
<% String attachmentFileName = (String)
request.getAttribute("attachmentFileName"); %>
```

- リクエストスコープからファイル名を取得する。

```
<% if (attachmentFileName != null && !attachmentFileName.isEmpty()) { %>
```

- ファイル名が null でも空文字でもない場合。

```
<%= attachmentFileName %>
```

- ファイル名を表示。

<% } else { %>
なし
<% } %>

- ファイルが無ければ「なし」と表示する。

</p>

- 添付ファイル表示の段落終了。

- display-info の終了。

送信フォーム

<form action="../inquiry" method="post">

- InquiryServlet に送信する form。

<input type="hidden" name="action" value="complete">

- action パラメータを hidden で送信する。値は "complete"。

- ボタンのグループ化。

<input type="submit" value="送信">

- 「送信」ボタン。

<a href="../inquiry" class="button secondary">戻る</a>

- 戻るリンク。

- ボタングループ終了。

</form>

- form 終了。

- コンテナ終了。

</body>
</html>

- HTML の終了。

complete.jsp 全ソース

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <title>お問い合わせ完了</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="container">
        <h1>お問い合わせ完了</h1>
        <p class="success-message">お問い合わせいただき、誠にありがとうございます。内容を確認後、改めてご連絡させていただきます。</p>
        <div class="button-group">
            <a href="../inquiry" class="button">トップに戻る</a>
            <a href="../inquiry?action=history" class="button secondary">お問い合わせ履歴を見る</a>
        </div>
    </div>
</body>
</html>
```

■complete.jsp 解説

以下、全行解説する。

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

- JSP のページディレクティブ。
- レスポンスの Content-Type を text/html に設定し、文字コードを UTF-8 にする。

<!DOCTYPE html>

- HTML5 の宣言。

<html lang="ja">

- HTML タグの開始。言語は日本語。

<head>

- head タグ開始。

<meta charset="UTF-8">

- HTML の文字エンコーディングを UTF-8 に設定。

<title>お問い合わせ完了</title>

- ブラウザのタブに表示されるページタイトル。

<link rel="stylesheet" href="../style.css">

- CSS ファイルを読み込む。../ は jsp フォルダから 1 つ上へ戻る。

</head>

- head タグ終了。

<body>

- body 開始。

- CSS の中央寄せレイアウト用のコンテナ。

<h1>お問い合わせ完了</h1>

- ページの見出し。

お問い合わせいただき、誠にありがとうございます。内容を確認後、改めてご連絡させていただきます。

- 完了メッセージ。
- success-message クラスで CSS による緑色のスタイルを適用。

- ボタンをまとめるための div。

<a href="../inquiry" class="button">トップに戻る</a>

- トップページ（フォーム画面）へ戻るリンク。
- CSS クラス button を適用。

<a href="../inquiry?action=history" class="button secondary">お問い合わせ履歴を見る</a>

- 履歴画面へのリンク。
- secondary クラスで色分け。

- button-group div 終了。

- container div 終了。

</body>
</html>

- HTML の終了。

inquiry_history.jsp 全ソース

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="UTF-8">
    <title>お問い合わせ履歴</title>
    <link rel="stylesheet" href="../style.css">
</head>
<body>
    <div class="container">
        <h1>お問い合わせ履歴</h1>

        <c:choose>
            <c:when test="${not empty inquiries}">
                <c:forEach var="inquiry" items="${inquiries}" varStatus="status">
                    <div class="inquiry-item">
                        <h3>お問い合わせ #${status.count}</h3>
                        <p><strong>名前:</strong> <c:out value="${inquiry.name}"/></p>
                        <p><strong>メールアドレス:</strong> <c:out value="${inquiry.email}"/></p>
                        <p><strong>内容:</strong></p>
                        <pre><c:out value="${inquiry.content}"/></pre>
                        <p><strong>添付ファイル:</strong>
                            <c:choose>
                                <c:when test="${not empty inquiry.attachmentFileName}">
                                    <a href="../uploads/<c:out value='${inquiry.attachmentFileName}'/>" target="_blank">
                                        <c:out value="${inquiry.attachmentFileName}"/>
                                    </a>
                                </c:when>
                                <c:otherwise>
                                    なし
                                </c:otherwise>
                            </c:choose>
                        </p>
                        <p><strong>ステータス:</strong> <c:out value="${inquiry.status}"/></p>

                        <form action="../inquiry" method="post" style="display:inline;">
                            <input type="hidden" name="action" value="updateStatus">
                            <input type="hidden" name="index" value="${status.index}">
                            <label for="status-${status.index}">ステータス変更:</label>
                            <select id="status-${status.index}" name="newStatus">
                                <option value="新規" <c:if test="${inquiry.status == '新規'}">selected</c:if>>新規</option>
                                <option value="対応中" <c:if test="${inquiry.status == '対応中'}">selected</c:if>>対応中</option>
                                <option value="完了" <c:if test="${inquiry.status == '完了'}">selected</c:if>>完了</option>
                            </select>
                            <input type="submit" value="更新" class="button secondary">
```

```
                    </form>
                </div>
                <hr>
            </c:forEach>
        </c:when>
        <c:otherwise>
            <p>まだお問い合わせはありません。</p>
        </c:otherwise>
    </c:choose>

    <div class="button-group">
        <a href="../inquiry" class="button secondary">お問い合わせフォームに戻る</a>
    </div>
  </div>
</body>
</html>
```

■inquiry_history.jsp 解説

<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

- JSP の基本設定。
- 出力コンテンツタイプと文字エンコーディングを指定。

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

- JSTL (JSP Standard Tag Library) の Core タグライブラリを使うための宣言。
- prefix="c" で c:xxx タグが使えるようになる。

<!DOCTYPE html>

- HTML5 の宣言。

<html lang="ja">

- HTML 開始。言語は日本語。

<head>

- head タグ開始。

<meta charset="UTF-8">

- HTML の文字エンコーディングを指定。

<title>お問い合わせ履歴</title>

- ブラウザタブに表示されるタイトル。

<link rel="stylesheet" href="../style.css">

- CSS ファイルを読み込む。
- ../ は jsp/ フォルダから 1 つ上へ戻る。

</head>

- head タグ終了。

<body>

- body 開始。

- コンテンツを中央寄せする div。

<h1>お問い合わせ履歴</h1>

- ページタイトル。

---

c:choose で条件分岐

<c:choose>

- 条件分岐を開始。

<c:when test="${not empty inquiries}">

- inquiries が空でない場合の処理。

---

c:forEach で inquiries を繰り返す

<c:forEach var="inquiry" items="${inquiries}" varStatus="status">

- inquiries の要素を 1 件ずつ取り出す。
- status には繰り返しの情報（index, count など）が入る。

- 問い合わせ情報の表示領域。

<h3>お問い合わせ #${status.count}</h3>

- 問い合わせ番号を表示。

名前: <c:out value="${inquiry.name}"/>

- 名前を出力。

メールアドレス: <c:out value="${inquiry.email}"/>

- メールアドレスを出力。

内容:

- 内容ラベル。

<pre><c:out value="${inquiry.content}"/></pre>

- 内容を表示。
- <pre> により改行を維持。

添付ファイル表示

添付ファイル:

- 添付ファイル欄開始。

<c:choose>

- 添付ファイルの有無で分岐。

<c:when test="${not empty inquiry.attachmentFileName}">

- ファイル名がある場合。

<a href="../uploads/<c:out value='${inquiry.attachmentFileName}'/>"
target="_blank">
<c:out value="${inquiry.attachmentFileName}"/>
</a>

- ファイル名をリンクで表示。
- target="_blank" で別タブで開く。

</c:when>
<c:otherwise>
なし
</c:otherwise>
</c:choose>

- ファイルが無ければ「なし」と表示。

- 添付ファイル欄終了。

ステータス表示

ステータス: <c:out value="${inquiry.status}"/>

- ステータスを表示。

ステータス変更フォーム

<form action="../inquiry" method="post" style="display:inline;">

- InquiryServlet へ POST する form。

<input type="hidden" name="action" value="updateStatus">

- action パラメータを hidden で送る。

<input type="hidden" name="index" value="${status.index}">

- どの Inquiry を更新するか index を送る。

<label for="status-${status.index}">ステータス変更:</label>

- ステータス変更のラベル。

<select id="status-${status.index}" name="newStatus">

- ステータスを選択する select ボックス。

ステータスの選択肢

<option value="新規" <c:if test="${inquiry.status == '新規'}">selected</c:if>>新規</option>

- 「新規」を選択肢として表示。
- 現在の status が「新規」なら selected。

<option value="対応中" <c:if test="${inquiry.status == '対応中'}">selected</c:if>>対応中</option>

- 「対応中」の選択肢。

<option value="完了" <c:if test="${inquiry.status == '完了'}">selected</c:if>>完了</option>

- 「完了」の選択肢。

</select>

- select 終了。

<input type="submit" value="更新" class="button secondary">

- ステータス更新ボタン。

</form>

- form 終了。

<hr>

- 1 件分の表示終了。区切り線。

</c:forEach>

- forEach 終了。

データがない場合

</c:when>
<c:otherwise>
まだお問い合わせはありません。
</c:otherwise>
</c:choose>

- inquiries が空ならメッセージ表示。

戻るリンク

<a href="../inquiry" class="button secondary">お問い合わせフォームに戻る</a>

- トップページへのリンク。

</body>
</html>

- HTML 終了。

style.css 全ソース

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    margin: 0;
    padding: 20px;
    background-color: #eef2f7;
    color: #333;
    line-height: 1.6;
}

.container {
    max-width: 600px;
    margin: 30px auto;
    background-color: #ffffff;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h1 {
    color: #2c3e50;
    text-align: center;
    margin-bottom: 30px;
    font-size: 2em;
}

form p {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
    color: #555;
}

input[type="text"],
input[type="email"],
textarea {
    width: calc(100% - 24px);
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1em;
    box-sizing: border-box;
    transition: border-color 0.3s ease;
}
```

```
input[type="text"]:focus,
input[type="email"]:focus,
textarea:focus {
    border-color: #007bff;
    outline: none;
}

textarea {
    resize: vertical;
    min-height: 100px;
}

.button-group {
    text-align: center;
    margin-top: 30px;
}

input[type="submit"],
.button {
    background-color: #007bff;
    color: white;
    padding: 12px 25px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 1.1em;
    transition: background-color 0.3s ease, transform 0.2s ease;
    display: inline-block;
    margin: 0 10px;
}

input[type="submit"]:hover,
.button:hover {
    background-color: #0056b3;
    transform: translateY(-2px);
}

.button.secondary {
    background-color: #6c757d;
}

.button.secondary:hover {
    background-color: #5a6268;
}
```

```
.error-message {
    color: #dc3545;
    font-size: 0.9em;
    margin-top: 5px;
    display: block;
}

.success-message {
    color: #28a745;
    font-size: 1.1em;
    text-align: center;
    margin-top: 20px;
    padding: 15px;
    background-color: #d4edda;
    border: 1px solid #c3e6cb;
    border-radius: 5px;
}

.display-info p {
    background-color: #f8f9fa;
    padding: 10px 15px;
    border-left: 4px solid #007bff;
    margin-bottom: 10px;
    border-radius: 4px;
}

.display-info p strong {
    color: #2c3e50;
}

pre {
    background-color: #f8f9fa;
    padding: 15px;
    border: 1px solid #e9ecef;
    border-radius: 6px;
    white-space: pre-wrap;
    word-wrap: break-word;
    font-family: 'Courier New', Courier, monospace;
    font-size: 0.95em;
}

a {
    color: #007bff;
    text-decoration: none;
    transition: color 0.3s ease;
}
```

```
a:hover {
    color: #0056b3;
    text-decoration: underline;
}
```

■style.css 全行解説

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    margin: 0;
    padding: 20px;
    background-color: #eef2f7;
    color: #333;
    line-height: 1.6;
}
```

- ページ全体のフォントや背景色を設定。
- line-height で行間を広くして読みやすくしている。

```
.container {
    max-width: 600px;
    margin: 30px auto;
    background-color: #ffffff;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}
```

- 中央寄せのボックスデザイン。
- box-shadow で影を付けて立体感を出す。

```
h1 {
    color: #2c3e50;
    text-align: center;
    margin-bottom: 30px;
    font-size: 2em;
}
```

- 見出しの色、中央寄せ、サイズ調整。

```
form p {
    margin-bottom: 20px;
}
```

- form 内の段落の余白。

```
label {
    display: block;
    margin-bottom: 8px;
    font-weight: bold;
    color: #555;
}
```

- ラベルを縦並びにする。
- 太字で視認性を上げる。

```
input[type="text"],
input[type="email"],
textarea {
    width: calc(100% - 24px);
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1em;
    box-sizing: border-box;
    transition: border-color 0.3s ease;
}
```

- テキスト入力と textarea の共通スタイル。
- calc で左右の余白を計算。

```
input[type="text"]:focus,
input[type="email"]:focus,
textarea:focus {
    border-color: #007bff;
    outline: none;
}
```

- フォーカス時に青枠で強調。

```
textarea {
    resize: vertical;
    min-height: 100px;
}
```

- テキストエリアを縦方向にのみサイズ変更可能に。

```
.button-group {
    text-align: center;
    margin-top: 30px;
}
```

- ボタンを中央に並べる。

```
input[type="submit"],
.button {
    background-color: #007bff;
    color: white;
    padding: 12px 25px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 1.1em;
    transition: background-color 0.3s ease, transform 0.2s ease;
    display: inline-block;
    margin: 0 10px;
}
```

- 共通ボタンスタイル。
- hover 時の変化も後で定義。

```
input[type="submit"]:hover,
.button:hover {
    background-color: #0056b3;
    transform: translateY(-2px);
}
```

- ホバー時に色を濃くし、少し上に浮かせる。

```
.button.secondary {
    background-color: #6c757d;
}
```

- secondary ボタンの色をグレー系に。

```
.button.secondary:hover {
    background-color: #5a6268;
}
```

- secondary ボタンのホバー時の色。

```
.error-message {
    color: #dc3545;
    font-size: 0.9em;
    margin-top: 5px;
    display: block;
}
```

- エラーメッセージは赤色。

```
.success-message {
    color: #28a745;
    font-size: 1.1em;
    text-align: center;
    margin-top: 20px;
    padding: 15px;
    background-color: #d4edda;
    border: 1px solid #c3e6cb;
    border-radius: 5px;
}
```

- 完了メッセージを緑系で強調。

```
.display-info p {
    background-color: #f8f9fa;
    padding: 10px 15px;
    border-left: 4px solid #007bff;
    margin-bottom: 10px;
    border-radius: 4px;
}
```

- 確認画面などの情報表示を枠で強調。

```
.display-info p strong {
    color: #2c3e50;
}
```

- 強調部分の文字色を濃くする。

```
pre {
    background-color: #f8f9fa;
    padding: 15px;
    border: 1px solid #e9ecef;
    border-radius: 6px;
    white-space: pre-wrap;
    word-wrap: break-word;
    font-family: 'Courier New', Courier, monospace;
    font-size: 0.95em;
}
```

- 〈pre〉タグの表示スタイル。
- 改行を維持しつつ、背景色で目立たせる。

```
a {
    color: #007bff;
    text-decoration: none;
    transition: color 0.3s ease;
}
```

- リンクの基本スタイル。

```
a:hover {
    color: #0056b3;
    text-decoration: underline;
}
```

- リンクにホバーしたときのスタイル。

第３章　学習ポイントまとめ

この章では、今回のプロジェクトで学んだ技術のポイントを、初心者向けにやさしく整理します。
本プロジェクトは「お問い合わせフォーム」という題材を通して、以下の多くの技術を学びます。

まず、**JavaBeans の作成方法**です。
JavaBeans とは、データを管理するためのクラスで、フィールドを private にし、getter/setter を持つのが特徴です。
今回の Inquiry.java は、お問い合わせの名前、メールアドレス、内容、添付ファイル名、ステータスなどを保持する役割を果たします。

次に、**DAO パターン**です。
データ操作をまとめたクラスが DAO（Data Access Object）です。
InquiryDAO.java は、お問い合わせ情報を保存したり取得したりする機能を担当します。
このように責務を分けることで、保守性が高まります。

そして、**サーブレット開発**も大切です。
サーブレットはブラウザから送られてくるリクエストを受け取り、処理してレスポンスを返すプログラムです。
今回の InquiryServlet.java では、doGet() と doPost() というメソッドを使い、画面表示やフォームの処理を行っています。

また、**ファイルアップロード**も学びました。
サーブレットでファイルを受け取るには @MultipartConfig というアノテーションを使い、容量制限などを設定します。

さらに、**セッション管理**も重要です。
一度入力したフォームの情報を確認画面に渡すために、HttpSession を使いました。
これで別のリクエストでも情報を保持できます。

JSTL の活用も学びました。
JSTL は JSP のタグライブラリで、<c:out> で値を表示したり、<c:forEach> で繰り返し処理ができたりします。
これにより、JSP の中で Java コードをほとんど書かずに済みます。

また、**クライアントサイドバリデーション**を JavaScript で実装しました。
これにより、サーバに送信する前に、必須項目の入力漏れをチェックできます。
しかし、ブラウザのチェックだけでは安心できないので、**サーバサイドバリデーション**もサーブレットで行っています。
もし入力に不備があれば、エラーメッセージを画面に表示します。

そして、今回特に注目したのが、**CAPTCHA の実装**です。
これはスパム対策の仕組みで、ランダムな計算問題を出題し、その答えをセッションに保存し、正しく入力されたかを確認します。
こうして、Web アプリ開発の基礎的な要素を一通り学ぶことができました。

■第４章　JSP ファイルの役割再整理

ここでは、画面ごとにどんな役割があるかをまとめます。

- index.jsp
  - 最初に表示される画面です。
  - 入力フォームを表示し、JSTL を使って前回の入力値を保持したり、エラーメッセージを表示します。
  - JavaScript で入力内容の簡易チェックも行います。
- confirm.jsp
  - 入力した内容を確認する画面です。
  - 入力情報を表示し、送信するか、修正するかを選べます。
- complete.jsp
  - お問い合わせ送信が完了したことを知らせる画面です。
  - トップへ戻るリンクや履歴画面へのリンクがあります。
- inquiry_history.jsp
  - 送信したお問い合わせの履歴を一覧で表示します。
  - ステータスの変更もできる管理的な画面です。

このように、画面を分けることでユーザー操作の流れがわかりやすくなり、システムも整理されます。

■第５章　CSS のデザインポイント

Web アプリは見た目もとても大事です。
今回のプロジェクトで使った CSS のポイントを解説します。
まず、.container クラス で画面全体を中央に配置しています。
見やすく整理されたレイアウトになります。
ボタンは基本的に青を基調とし、操作性を高めています。
履歴画面へのリンクや戻るボタンなど、重要度が低いボタンにはグレー（secondary）を使ってメリハリをつけています。
.error-message は赤色で、入力エラーを目立たせます。
一方、.success-message は緑色で、操作成功を知らせます。
.display-info は確認画面や履歴画面で入力情報をきれいに整えるために使っています。
枠や背景色で情報を区切り、読みやすさを高めています。
<pre> タグを使うことで、入力した内容の改行を保ったまま表示できるようにしました。
背景色を変えて差別化し、ユーザーが入力内容を確認しやすくしています。
box-shadow を使うことで、画面に立体感を持たせ、デザイン性を高めています。
レスポンシブ対応は最低限ですが、今回のコードは簡単に拡張できるようになっています。
スマホ対応などもすぐに行えるデザインです。

■第６章　処理の流れ（動作フロー）

ここでは、アプリ全体の流れを初心者でもわかるように説明します。

【ステップ 1】
ユーザーが /inquiry にアクセスします。
→ サーブレットの doGet() が呼ばれ、index.jsp が表示されます。
→ 同時に CAPTCHA の計算問題が作られ、画面に表示されます。

【ステップ 2】
ユーザーがフォームに入力し、送信ボタンを押します（POST リクエスト）。
→ doPost() メソッドが呼ばれます。
→ 入力内容がサーバ側でチェックされます（サーバサイドバリデーション）。
→ CAPTCHA の答えも確認されます。
もし入力に不備があれば、再び index.jsp が表示され、エラーメッセージが出ます。
問題なければ confirm.jsp に進みます。

【ステップ 3】
confirm.jsp が表示され、入力内容が確認できます。
「送信」ボタンを押すと、POST で action=complete が送信されます。

【ステップ 4】
InquiryServlet が action=complete を受け取り、
セッションから Inquiry のデータを取り出し、DAO に保存します。
保存が終わると complete.jsp が表示されます。

【ステップ 5】
ユーザーが「履歴を見る」リンクを押すと
GET /inquiry?action=history が呼ばれます。
InquiryServlet は全履歴を取得し、inquiry_history.jsp に表示します。
履歴画面からステータス変更も行えます。

このように、今回のアプリは「フォーム → 確認 → 完了 → 履歴」という典型的な Web アプリの流れを体験できます。

■第７章　次の学びへのステップ

この「お問い合わせフォーム」は初級レベルとしてはとても良い題材です。
ただし、実際のビジネスアプリではまだ機能が足りません。
次のような課題に挑戦すると、より実践的なスキルが身につきます。

■JDBC や JPA による DB 永続化

- 今回のアプリはメモリ上にデータを保存しているため、サーバを止めるとデータは消えます。
- データベースに保存すれば永続的にデータを保持できます。

■ログイン認証機能

- ユーザーごとにデータを管理するには、認証が必要です。
- 「誰がいつ問い合わせたか」まで管理できるようになります。

■メール送信機能

- 問い合わせを送信したら、自動で「受付メール」を送るのが現場では当たり前です。
- JavaMail API を使うなど、メール送信を学ぶ良い機会です。

■ログ出力（Log4j, SLF4J）

- エラーや操作履歴をファイルに残す仕組みも重要です。
- 問題の原因調査やセキュリティにも役立ちます。

■AJAX による非同期通信

- 画面全体をリロードせず、部分的に更新できる技術です。
- 履歴画面のステータス更新を非同期でやればユーザ体験が向上します。

■バリデーションフレームワーク（Bean Validation）

- サーブレットに長いバリデーションコードを書く代わりに、アノテーションで簡単に入力チェックできます。

これらはすべて実務で使われる技術です。
ぜひ挑戦してみましょう！

■第８章　まとめ

今回の教材で作った「お問い合わせフォーム」には、動的 Web アプリ開発の大切なポイントがすべて詰まっています。

- JavaBeans を使ってデータを管理する方法
- サーブレットでリクエストを受け取り、レスポンスを返す仕組み
- ファイルをアップロードする方法
- セッションを利用してデータを一時的に保存する技術
- JSP と JSTL を使った画面作成
- サーバとブラウザ双方で行うバリデーション
- CAPTCHA を使ったスパム対策
- 見やすく、使いやすい画面のデザイン技術